Практическая работа 1 Азаров
Блок 1 - namespaces
Требуется просмотреть namespace-ы текущего shell-процесса. Это делается командой "ls -la /proc/$$/ns/"

```
zoker@zoker:~$ ls -la /proc/$$/ns/
total 0
dr-x--x--x 2 zoker zoker 0 Mar 24 23:49 .
dr-xr-xr-x 9 zoker zoker 0 Mar 24 23:49 ..
lrwxrwxrwx 1 zoker zoker 0 Mar 24 23:49 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 zoker zoker 0 Mar 24 23:49 ipc -> 'ipc:[4026531839]'
lrwxrwxrwx 1 zoker zoker 0 Mar 24 23:49 mnt -> 'mnt:[4026531832]'
lrwxrwxrwx 1 zoker zoker 0 Mar 24 23:49 net -> 'net:[4026531833]'
lrwxrwxrwx 1 zoker zoker 0 Mar 24 23:49 pid -> 'pid:[4026531836]'
lrwxrwxrwx 1 zoker zoker 0 Mar 24 23:49 pid_for_children -> 'pid:[4026531836]'
lrwxrwxrwx 1 zoker zoker 0 Mar 24 23:49 time -> 'time:[4026531834]'
lrwxrwxrwx 1 zoker zoker 0 Mar 24 23:49 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 zoker zoker 0 Mar 24 23:49 user -> 'user:[4026531837]'
lrwxrwxrwx 1 zoker zoker 0 Mar 24 23:49 uts -> 'uts:[4026531838]'
zoker@zoker:~$
```

Описание namespace-ов:

mnt (Mount): 4026531832
Отвечает за точки монтирования. Процесс видит свой собственный корень файловой системы и смонтированные устройства. Если сделать chroot или запустить в контейнере, то иноды будут другими

pid и pid_for_children: 4026531836
Отвечает за изоляцию дерева процессов. Процесс видит только своих «детей» и процессы в своем неймспейсе. pid_for_children показывает, в каком неймспейсе PID окажутся будущие дочерние процессы (обычно совпадает с pid, если процесс не вызвал unshare(CLONE_NEWPID) без передачи флага).

net (Network): 4026531833
Изолирует сетевой стек: интерфейсы, IP-адреса, маршруты, правила iptables. У вас сейчас инод 1833 — это, скорее всего, «глобальный» или «корневой» сетевой неймспейс хоста.

ipc (Inter-Process Communication): 4026531839
Изолирует механизмы межпроцессного взаимодействия System V и POSIX message queues. Если бы процессы находились в разных ipc, они не могли бы общаться через разделяемую память (shared memory) или семафоры.

uts (UNIX Time Sharing): 4026531838
Изолирует hostname и domainname. Благодаря этому в контейнере можно задать имя web-server-1, не меняя имя хостовой машины.

user (User): 4026531837

Изолирует пользователей и группы. Позволяет процессу внутри неймспейса иметь root (UID 0) для операций внутри этого неймспейса, но при этом на хосте этот процесс является обычным непривилегированным пользователем. Это основа безопасности контейнеров (rootless containers).

cgroup: 4026531835
Позволяет процессу иметь свой собственный иерархический путь в cgroups. Это нужно для изоляции ограничений ресурсов (CPU, память) для групп процессов.

time и time_for_children: 4026531834
Позволяет изолировать системные часы (монотонные, boot-time). Нужен для того, чтобы процессы внутри неймспейса «не знали», как долго работает хост, или чтобы корректно работали паузы/возобновления работы контейнеров.

Просмотрим все типы неймспейсов в системе командой "lsns"

```
zoker@zoker:~$ lsns
        NS TYPE   NPROCS   PID USER  COMMAND
4026531832 mnt        92  3025 zoker /usr/bin/pipewire
4026531833 net        98  3025 zoker /usr/bin/pipewire
4026531834 time      117  3025 zoker /usr/bin/pipewire
4026531835 cgroup    117  3025 zoker /usr/bin/pipewire
4026531836 pid       117  3025 zoker /usr/bin/pipewire
4026531837 user       97  3025 zoker /usr/bin/pipewire
4026531838 uts       117  3025 zoker /usr/bin/pipewire
4026531839 ipc        98  3025 zoker /usr/bin/pipewire
4026532760 user        1 19323 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026532761 mnt         0       root
4026532931 user        1  5733 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026532932 ipc         1  5733 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026532933 net         1  5733 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533001 mnt         0       root
4026533007 ipc         1 19323 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533008 net         1 19323 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533063 user        1 19380 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533064 mnt        23  4409 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533065 mnt         2  4042 zoker /snap/snapd-desktop-integration/343/usr/bin
4026533066 user        1  4630 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533067 ipc         1  4630 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533068 mnt         0       root
4026533070 user        1  4657 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533071 ipc         1  4657 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533072 net         1  4657 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533127 user        1  4667 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533128 net         1  4667 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533183 user        1  4886 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533184 ipc         1  4886 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533185 net         1  4886 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533240 user        1  5244 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533241 ipc         1  5244 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533242 net         1  5244 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533297 user        1  5392 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533298 ipc         1  5392 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533299 net         1  5392 zoker /snap/firefox/7967/usr/lib/firefox/firefox
4026533354 user        1  5395 zoker /snap/firefox/7967/usr/lib/firefox/firefox
```

В целом lsns выступает как некий паспортный стол для неймспейсов, потому что эта команда показывает все активные пространства неймспейсов.

Структура вывода lsns

NS TYPE — тип неймспейса (mnt, net, pid, user, ipc, uts, cgroup, time)
NPROCS — количество процессов, использующих этот неймспейс
PID — PID одного из процессов в этом неймспейсе (обычно первого попавшегося)
USER — владелец процесса
COMMAND — команда, которая запустила процесс

Запустим bash в новом PID namespace. Для этого нам понадобится команда sudo unshare --pid --fork --mount-proc /bin/bash

Описание каждой части команды:
sudo - Запуск от root (нужен для создания некоторых типов неймспейсов)
unshare - Утилита для запуска программы в новых неймспейсах
--pid - Создать новый PID namespace (изоляция процессов)
--fork - Создать новый процесс (fork) перед запуском программы
--mount-proc - Автоматически примонтировать /proc в новом неймспейсе
/bin/bash - Какую программу запустить внутри изоляции

```
zoker@zoker:~$ sudo unshare --pid --fork --mount-proc /bin/bash
[sudo] password for zoker:
Sorry, try again.
[sudo] password for zoker:
Sorry, try again.
[sudo] password for zoker:
root@zoker:/home/zoker#
```

Убедимся что мы PID 1 echo "Мой PID: $$"

```
root@zoker:/home/zoker# echo "Мой PID: $$"
Мой PID: 1
root@zoker:/home/zoker#
```

Просмотрим запущенные процессы командой ps aux

```
root@zoker:/home/zoker# ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0  10212  4448 pts/2    S    00:09   0:00 /bin/bash
root           8  0.0  0.0  13492  4536 pts/2    R+   00:13   0:00 ps aux
root@zoker:/home/zoker# top
```

Запущено два процесса всего, потому что мы запустили баш в контейнере. При выходе этот контейнер просто напросто исчезнет.

Выйдем в глобальное окружение командой exit

Создадим изолированное сетевое окружение командой sudo unshare --net /bin/bash просмотрим список сетевых интерфейсов командой ip link show. Мы увидим, что есть только loopback-интерфейс, и тот неактивен, а значит изоляция работает.

```
zoker@zoker:~$ sudo unshare --net /bin/bash
root@zoker:/home/zoker# ip link show
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
root@zoker:/home/zoker#
```

Выйдем из окружения так же командой exit

Блок 2 - cgroups
Найдем иерархию cgroup v2: ls /sys/fs/cgroup/

Cgroups - это важнейший механизм для ограничения и учёта ресурсов процессов.

```
zoker@zoker:~$ ls /sys/fs/cgroup/
cgroup.controllers      cgroup.stat             cpuset.cpus.isolated   dev-mqueue.mount   io.cost.qos     memory.numa_stat        misc.capacity                  sys-kernel-config.mount
cgroup.max.depth        cgroup.subtree_control  cpuset.mems.effective  dmem.capacity      io.pressure     memory.pressure         misc.current                   sys-kernel-debug.mount
cgroup.max.descendants  cgroup.threads          cpu.stat               dmem.current       io.prio.class   memory.reclaim          misc.peak                      sys-kernel-tracing.mount
cgroup.pressure         cpu.pressure            cpu.stat.local         init.scope         io.stat         memory.stat             proc-sys-fs-binfmt_misc.mount  system.slice
cgroup.procs            cpuset.cpus.effective   dev-hugepages.mount    io.cost.model      machine.slice   memory.zswap.writeback  sys-fs-fuse-connections.mount  user.slice
zoker@zoker:~$
```

Создадим свою cgroup и ограничим CPU до 20%:
sudo mkdir /sys/fs/cgroup/mytest
echo "20000 100000" | sudo tee /sys/fs/cgroup/mytest/cpu.max

```
zoker@zoker:~$ sudo mkdir /sys/fs/cgroup/mytest
echo "20000 100000" | sudo tee /sys/fs/cgroup/mytest/cpu.max
20000 100000
zoker@zoker:~$
```

Запустим нагрузку и поместим ее в cgroup:
stress-ng --cpu 2 --timeout 30s & echo $! | sudo tee /sys/fs/cgroup/mytest/cgroup.procs

```
zoker@zoker:~$ stress-ng --cpu 2 --timeout 30s &
echo $! | sudo tee /sys/fs/cgroup/mytest/cgroup.procs
[1] 23344
stress-ng: info:  [23344] setting to a 30 secs run per stressor
23344
stress-ng: info:  [23344] dispatching hogs: 2 cpu
zoker@zoker:~$
```

Проверим, что лимит применился и посмотрим в top:
cat /sys/fs/cgroup/mytest/cpu.stat

```
zoker@zoker:~$ cat /sys/fs/cgroup/mytest/cpu.stat
usage_usec 6037933
user_usec 6013746
system_usec 24186
nice_usec 0
core_sched.force_idle_usec 0
nr_periods 313
nr_throttled 301
throttled_usec 54079047
nr_bursts 0
burst_usec 0
```

top

```
zoker@zoker:~$ top

top - 00:34:22 up 51 min,  1 user,  load average: 1.53, 1.52, 1.36
Tasks: 393 total,   5 running, 388 sleeping,   0 stopped,   0 zombie
%Cpu(s): 19.2 us,  0.4 sy,  0.0 ni, 80.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  23732.1 total,  13420.8 free,   5013.4 used,   5931.9 buff/cache
MiB Swap:   8192.0 total,   8192.0 free,      0.0 used.  18718.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  24026 zoker     20   0   56832   5692   2552 R 100.0   0.0   0:24.35 stress-ng-cpu
  24025 zoker     20   0   56832   5680   2540 R  99.3   0.0   0:24.32 stress-ng-cpu
  24036 zoker     20   0   56832   5688   2548 R  10.0   0.0   0:02.06 stress-ng-cpu
  24035 zoker     20   0   56832   5688   2548 R   9.6   0.0   0:02.10 stress-ng-cpu
```

Скриншот лимита (cat /sys/fs/cgroup/mytest/cpu.max):

```
zoker@zoker:~$ cat /sys/fs/cgroup/mytest/cpu.max
20000 100000
zoker@zoker:~$
```

Уберем нагрузку, убив процесс:
sudo kill $(cat /sys/fs/cgroup/mytest/cgroup.procs)
Также можно просто подождать 30 секунд, по истечении времени процесс умрет сам

Блок 3 - chroot
создадим минимальный rootfs:
mkdir -p /tmp/myroot/{bin,lib,lib64,proc,dev}
Создаёт директорию /tmp/myroot, которая станет "корнем" новой системы
Внутри создаёт поддиректории:
bin — для исполняемых файлов
lib и lib64 — для библиотек
proc — точка монтирования procfs
dev — для устройств

Скопируем баш и его зависимости:
cp /bin/bash /tmp/myroot/bin/

cp /bin/ls /tmp/myroot/bin/
Скопируем нужные библиотеки:
ldd /bin/bash

Создадим нужные директории для библиотек:
mkdir -p /tmp/myroot/lib/x86_64-linux-gnu
mkdir -p /tmp/myroot/lib64
Затем скопируем их все:
cp /lib/x86_64-linux-gnu/libtinfo.so.6 /tmp/myroot/lib/x86_64-linux-gnu/
cp /lib/x86_64-linux-gnu/libc.so.6 /tmp/myroot/lib/x86_64-linux-gnu/
cp /lib64/ld-linux-x86-64.so.2 /tmp/myroot/lib64/

Скопируем все необходимые зависимости одной командой: for dep in $(ldd /bin/bash | awk '/=>/ {print $3}'); do
cp --parents "$dep" /tmp/myroot/
done
cp /lib64/ld-linux-x86-64.so.2 /tmp/myroot/lib64/
скопируем также динамический загрузчик
Копирует динамический загрузчик (ELF interpreter)
Это самая важная библиотека — она запускает все динамически слинкованные программы Почему отдельно:
ldd может не показать загрузчик в выводе (он в первой строке linux-vdso.so.1)
Без него программы не запустятся

```
zoker@zoker:~$ ldd /bin/bash
        linux-vdso.so.1 (0x0000782f4cade000)
        libtinfo.so.6 => /lib/x86_64-linux-gnu/libtinfo.so.6 (0x0000782f4c922000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000782f4c600000)
        /lib64/ld-linux-x86-64.so.2 (0x0000782f4cae0000)
zoker@zoker:~$ ^C
zoker@zoker:~$ mkdir -p /tmp/myroot/lib/x86_64-linux-gnu
mkdir -p /tmp/myroot/lib64
zoker@zoker:~$ cp /lib/x86_64-linux-gnu/libtinfo.so.6 /tmp/myroot/lib/x86_64-linux-
cp /lib/x86_64-linux-gnu/libc.so.6 /tmp/myroot/lib/x86_64-linux-gnu/
cp /lib64/ld-linux-x86-64.so.2 /tmp/myroot/lib64/
zoker@zoker:~$ for dep in $(ldd /bin/bash | awk '/=>/ {print $3}'); do
    cp --parents "$dep" /tmp/myroot/
done
cp /lib64/ld-linux-x86-64.so.2 /tmp/myroot/lib64/
zoker@zoker:~$
```

Войдем в chroot-окружение : sudo chroot /tmp/myroot /bin/bash

```
zoker@zoker:~$ sudo chroot /tmp/myroot /bin/bash
bash-5.2# ls /
bin  dev  lib  lib64  proc
bash-5.2# echo "\320\257 \320\262\320\275\321\203\321\202\321\200\320\270 chroot!"
Я внутри chroot!
bash-5.2# exit
exit
zoker@zoker:~$
```

Итог: Вот как работает контейнер — namespace + cgroup + chroot. Docker это автоматизирует.